Using Context to Share State

For our app, we're going to use two React hooks to help us share state. The first hook we're going to use is useContext, which lets us use a feature of React called a *context*. In React, contexts allow us to share global data among components without passing that data via props.

To use a context, we surround our code with a special JSX component called a *context provider*. The context provider is initialized with a value, and then any component inside that provider, no matter how many levels down, can use the useContext hook to give that component access to the data in the context.

The specific data we want to share in our context is a reducer function that will provide our common state and a dispatch function to allow us to update that state. We'll use a React hook called useReducer, which takes as an argument a function that implements the reducer pattern and provides some ease of use for accessing the reducer.

Changing our code to use a context and a reducer ends up being a significant refactoring of our React code, so let's outline what the current data flow looks like:

- Our existing data model sets a bunch of data at the Venue level, which is passed down to each Row.
- Each Row maintains a list of the statuses of its component seats and passes that information to each Seat.
- The Seat status is held in the Row because the status depends on the status of neighboring seats and the Row is where all that data is stored.
- When we click on a Seat, that data is passed back up to the Row, which updates the status and passes the status back down to the Seat.

If we add the subtotal calculator, which would be a sibling component to the